Multi-LLM Architectures & Eco-Responsible AI
Project - Multi-Agent, Multi-LLM RAG System for Digital Transformation Roadmap Generation

You are required to submit your deliverables on BOOSTCAMP, with respect to the deadline set by your instructor. Submit your own work. Cheating will not be tolerated and will be penalized.
This activity can be done in group (up to 2 members per group).

Theoretical Framework and Role of the Project
GenAI, RAG, Multi-Agent Systems and Multi-LLM Routing

Generative AI and LLMs: Foundations and Role in the Project

Language models (LLMs – Large Language Models) now form the core of generative artificial intelligence systems. They are capable of understanding natural language instructions, producing structured text, summarizing, analyzing, and performing guided reasoning.

In a digital transformation context, these capabilities make it possible to:
- analyze a business situation,
- interpret strategic documents,
- generate recommendations,
- build actionable plans.

In this project, LLMs are not used in isolation, but integrated into a complete intelligent system.

RAG (Retrieval-Augmented Generation) : Foundations and Use

RAG consists of combining:
- a retrieval phase (information search)
- with a generation phase (LLM)

The system retrieves relevant information from a corpus, injects it into the prompt, then generates a response grounded in this data.

This makes it possible to:
- reduce hallucinations,
- anchor responses in a reference framework,
- improve the reliability of results.

In this project, RAG is based exclusively on the provided documents (local RAG), as described in the subject.

Multi-Agent Systems: Role and Interest

A multi-agent system relies on the collaboration of several specialized agents.

Each agent has:
- a role,
- an objective,
- a specific responsibility.

In this project, reasoning is broken down into several steps:
- one agent analyzes the problem,
- one agent retrieves the information,
- one agent structures the analysis,
- one agent generates the roadmap,
- one agent evaluates quality.

This approach provides:
- better reasoning quality,
- modularity,
- better explainability.

Multi-LLM and Routing: Performance Optimization

Unlike traditional systems relying on a single model, this project introduces a multi-model architecture (Multi-LLM).

The principle is to use several models depending on the type of task.

Examples:
- a fast model for simple tasks (classification, extraction)
- a powerful model for complex tasks (reasoning, strategy)
- a local model for frequent tasks to reduce costs

Routing acts as the decision layer:
- it analyzes the prompt
- it selects the most suitable model

Objectives:
- improve performance
- reduce costs
- optimize latency

Overall System Architecture

The system to be designed follows an architecture of the following type:

The system you must design follows an intelligent processing chain, in which each block plays a precise role in transforming a user request into a relevant and exploitable response.

1. User Input

The process begins with a user request.
Example: “Our company wants to improve customer experience and reduce operational costs through digital transformation.”
At this stage:

- the input is unstructured
- it may be vague or incomplete
- it requires interpretation

The system must therefore understand the intent.

2. RAG (Information Search)

Before generating a response, the system searches the documents.

- Search within:
- frameworks (Why / What / How)
- Digital Transformation Canvas
- roadmap templates

Objective:

- retrieve relevant information
- avoid responding “randomly”

An enriched context is built and sent to the LLM.
This is the key difference: the model does not answer alone; it relies on knowledge.

3. Specialized Agents (Multi-Agent System)

The work is then decomposed among several agents. Each agent has a clear role:

- **Planner**: understands the problem and structures the objectives
- **Analyst**: analyzes according to frameworks (canvas, strategy, etc.)
- **Strategist / Generator**: proposes initiatives
- **Evaluator**: checks consistency

Why?

- one single LLM for limited reasoning
- several agents for more structured and reliable reasoning

We therefore move from a direct response to an organized thinking process.

4. Multi-LLM Routing (Decision layer)

At each step, the system chooses the right model.

Example:
- simple task: fast model (Gemini Flash)
- complex analysis: powerful model (Gemini Pro)
- repetitive tasks: local model (LLaMA)

Routing decides according to:
- task type
- complexity
- cost
- latency

The system becomes intelligent and optimized (performance / cost).

5. Final Generation (Final Output)

The system produces a structured final response.
Example of output:
- digital transformation roadmap
- initiatives
- objectives
- owners
- budget
- timeline
- KPIs

The response is:
- based on documents (RAG)
- structured (agents)
- optimized (routing)

It is not free text: it is a result that can be exploited in a business context.

Overall Role of the Project

This project is not only about generating text. It is about designing a complete intelligent system, capable of:
- understanding a business context
- exploiting a documentary corpus
- structuring a strategic analysis
- generating an actionable roadmap
- optimizing LLM calls

You adopt the posture of an AI Architect / AI Engineer, capable of designing a complete solution.

Objectives of the Project

At the end of this project, you will be able to:

- design a complete RAG architecture
- implement a multi-agent system
- use several LLMs within the same architecture
- implement a routing mechanism
- optimize API calls and costs
- structure a complete AI pipeline
- develop an exploitable application

Project Specifications (Cahier des Charges)

Project Context

Digital transformation is now a major strategic issue for companies. It is not limited to the adoption of digital tools, but implies a deeper evolution of processes, business models, customer relationships, technological capabilities, governance, organizational culture, and value creation. The IMD/Cisco framework specifically emphasizes the idea that transformation must be thought of as a structured journey around three central questions: Why transform? What to transform? How to transform? (Wade, 2015).

The project you must carry out follows this logic. As indicated in the shared project document, the objective is to design an AI system based on LLMs, RAG, and a multi-agent architecture, capable of analyzing a documentary corpus on digital transformation and then proposing, from a business case, a structured digital transformation roadmap. The expected system is not limited to answering questions; it must understand the context of a company, retrieve relevant information from the corpus, reason according to a structured framework, generate transformation initiatives, and produce an exploitable roadmap.

This project must therefore not be approached as a simple prompting exercise. It is about designing a complete AI architecture, combining:

- a documentary knowledge base,
- a retrieval pipeline,
- several specialized agents,
- several language models,
- and a routing and optimization logic.

General Objective

The general objective of the project is to develop an MVP (Minimum Viable Product) capable of transforming a company description into a structured, coherent, and actionable digital transformation roadmap.

From a business case, the system must be capable of:

- understanding the stakes and transformation needs,
- exploiting the provided documentary frameworks,
- analyzing the company according to several complementary reading grids,
- proposing appropriate initiatives,
- organizing these initiatives into a roadmap logic,
- producing an exploitable output in a steering logic.

The expected final deliverable is therefore not generic text or a simple narrative summary, but a structured result that can be used to support strategic reflection or transformation scoping.

Reference Corpus and Frameworks to Be Used

The project relies on several reference documents that must be used as the system's reasoning foundation. The corpus includes four main frameworks / resources:

- Wade, M. (2015). Digital business transformation: A conceptual framework. Global Center for Digital Business Transformation.
- Peter, M. K. (2018). Digital transformation canvas: The 7 action fields of transformation, Best practice reference guide. University of Applied Sciences and Arts Northwestern Switzerland, School of Business. Available on www.marcpeter.com
- Elia, G., Solazzo, G., Lerro, A., Pigni, F., & Tucci, C. L. (2024). The digital transformation canvas: A conceptual framework for leading the digital transformation process. Business Horizons, 67, 381–398. https://doi.org/10.1016/j.bushor.2024.03.007
- Peter, M. K. (2024). The digital roadmap: Template and SME example (Alexandra's Bikeshop). DigitalProf.

IMD/Cisco Framework: Why transform? / What to transform? / How to transform? (Wade, 2015)

The document *Digital Business Transformation: A Conceptual Framework* defines digital transformation as a process of organizational change based on the use of digital technologies and digital business models to improve performance. It explicitly structures the transformation journey around three guiding questions:

- **Why transform?**
- **What to transform?**
- **How to transform?**

This framework is essential in the project because it makes it possible to:

- clarify the motivations for transformation,
- identify priority transformation objects,
- structure the logic of moving to action.

The same document also reminds us that transformation is not a state but a journey, and that it may be driven by customer expectations, competitors, emerging technologies, or growing pressure from industry disruption.

The Digital Transformation Canvas: Digital Transformation Canvas – 7 Action Fields (Peter, 2018)

Marc K. Peter's guide presents the Digital Transformation Canvas, organized around seven action fields:

1. Customer Centricity
2. New Technologies
3. Cloud and Data
4. Digital Business Development
5. Process Engineering
6. Digital Leadership & Culture
7. Digital Marketing

This framework is particularly valuable for structuring the company analysis in an operational way. It does not merely name dimensions; it also provides guiding questions that make it possible to conduct a workshop-type reflection, for example on customer segments, digital needs, available technologies, process automation, digital marketing strategies, or cultural and managerial change.

The same guide also specifies that digital transformation can be thought of as a strategic initiative implemented through several projects, following a simplified process including in particular:

- a **Maturity Analysis**,
- a **Strategic Analysis**,
- a **Strategy Development**,
- a **Roadmap & Implementation** stage,
- then **Change Management and Leadership** dimensions,
- and finally **Marketing and Continuous Optimisation**.

Recent Academic Framework: Strategic and Operational Canvas (Elia et al., 2024)

The article *The digital transformation canvas: A conceptual framework for leading the digital transformation process* complements the previous frameworks by proposing a more academic, structured, and systemic vision of digital transformation.

It identifies 11 key elements, grouped into 4 major categories:

- **Digital Transformation Strategy**
- **Digital Transformation Operational Pillars**
- **Digital Transformation Value**
- **Digital Transformation Pitfalls**

These categories include in particular:

- **Purpose**
- **Process, People, Platform, Partners, Project**
- **Product, Performance, Planet**
- **Protection, Privacy**

This framework makes it possible to go further than a simple functional analysis. It requires the integration of:

- the strategic purpose of the initiative,
- the operational pillars required for its implementation,
- the expected value,
- as well as protection and privacy-related risks and constraints.

It also specifies that these elements can be mobilized to represent and compare initiatives according to budget, timing, and risk constraints, which strongly reinforces the relevance of this framework for roadmap generation.

Business Input Document and Roadmap Generation (Peter, 2024)

The system must also be able to take as input one or more company-specific documents in order to generate a roadmap adapted to its specific context.

Within the pedagogical framework of this project, the example business document, such as **Alexandra's Bikeshop Digital Roadmap**, must not be considered as an output template to be mechanically reproduced. It must be interpreted as an input document representing the case of a real company, used to simulate and demonstrate how the system can work on a concrete example.

Recap :

The three shared documents / frameworks constitute the knowledge base / reference corpus.

The business document constitutes the business input to be analyzed;

- the generated roadmap must be a reasoned production, not an imitation of the input document.

In a more general logic, the system can be designed to accept several business documents or a single one. In this project, a single document is provided for pedagogical purposes, in order to limit experimental complexity and reduce the consumption related to free APIs.

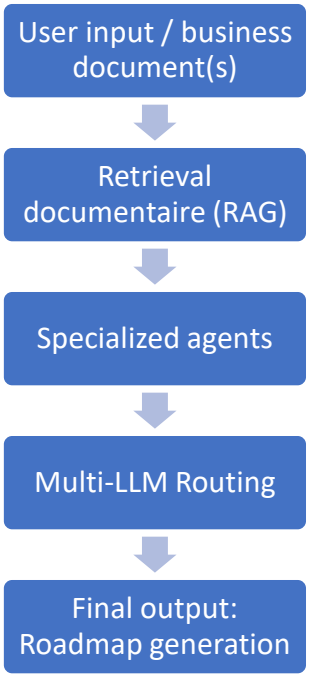
Project Problem Statement

You must answer the following problem statement:

How can an intelligent system be designed, based on LLMs, local RAG, a multi-agent architecture, and multi-LLM routing, capable of generating a credible digital transformation roadmap from a business case and a corpus of reference frameworks?

Expected Overall Architecture

The target architecture of the system follows the following logic:



User input / business document
The system must be able to receive:
- a business case written in free text,
- or a document describing the company, its stakes, objectives, and constraints.

This input may be vague, partial, or highly narrative. The system must be able to interpret it, extract useful elements from it, then make it a starting point for reasoning.

Document retrieval (RAG)
The system must then search the shared corpus for the most relevant passages to shed light on the business case. The initial project clearly specifies that the RAG will be local, based only on the provided files, with no external web resource.
The role of retrieval is:
- to anchor reasoning in documents,
- to limit hallucinations,
- to provide reliable context to agents and LLMs.

Specialized agents
The project relies on a reasoning decomposition logic already explicitly recommended in the source document: one agent plans, one agent retrieves information, one agent analyzes according to a framework, one agent generates the roadmap, and one agent checks consistency.

Multi-LLM routing
At each stage, the system must be able to select the most suitable model for the task. This routing layer acts as a decision layer:
- simple task: fast / economical model,
- complex task: more powerful model,
- repetitive or local tasks: open-source local model if possible.

Final output: Roadmap generation
The final output must be a structured, coherent, exploitable roadmap aligned with the transformation frameworks used.

Methodological Guidelines and Good Practices

Here are some methodological, technical, and prototyping recommendations integrating:
- the development of a complete prototype,
- a lightweight full stack logic,
- the free choice of development environment (Streamlit recommended),
- the importance of structuring the code, versioning it, designing an exploitable interface, and possibly industrializing it via API.

Recommended Methodological Approach

You must adopt a step-by-step progression, consistent with the logic of the project, by gradually building your system from simple building blocks toward a more complete architecture:

- simple LLM call,
- first chain,
- RAG pipeline,
- specialized agents,
- multi-agent orchestration,
- multi-LLM integration,
- routing,
- optimization,
- exploitable final prototype.

It is strongly recommended to:

- test each component separately before integration,
- version your work regularly with Git,
- keep input / output examples to validate the system's behavior,
- systematically document your technical and methodological choices,
- gradually build a modular and maintainable solution.

Methodological, Technical, and Prototyping Recommendations

The success of this project does not depend only on the quality of the responses generated by the LLMs, but on your ability to design a complete, structured, maintainable, and exploitable prototype, close to a real AI product. The objective is to make you adopt the posture of an AI Engineer / AI Product Builder, capable of thinking at the same time about:

- AI architecture,
- code organization,
- user interface,
- component integration,
- and, if possible, industrialization of the system.

The recommendations below are intended to guide you toward a more robust, clearer, and more professional solution.

1. Develop a real prototype, and not only a technical pipeline

Your project must not be limited to a series of scripts or a notebook demonstrating a few LLM calls. You must aim to develop a functional prototype, that is, a system that can actually be used to submit a business case, launch the analysis, and obtain a roadmap generated by the system.

This means that your solution should ideally include:

- a well-structured processing logic,
- an interface or a clear entry point for the user,
- a modular code organization,
- and a clear separation between business logic, AI logic, and the presentation layer.

The objective is not only to show that it works, but to show that your solution is designed like a mini software product.

2. Choose the development environment of your preference, according to your experience

You are free to choose the development environment that best matches your preferences, your level, and your previous experience. No single framework is imposed.
You may, for example, choose:

- Streamlit
- Dash / Plotly
- Flask
- FastAPI and a simple front-end
- React and Python backend
- or any other environment consistent with your project

What matters is not choosing the most complex tool, but choosing an environment that you master well enough to produce a functional, coherent, and well-structured solution.

For this project, **Streamlit is strongly recommended, especially for students who do not have advanced full-stack development skills**, for several reasons:

- it is an open-source framework,
- it is designed for data scientists and AI engineers,
- it allows you to quickly build an interactive Python interface,
- it does not require advanced front-end development skills,
- it facilitates the prototyping of interactive AI applications,
- it makes it easy to display:
- text,
- tables,
- forms,
- logs,
- structured outputs,
- and visualizations.

Streamlit is particularly suitable if you want to quickly build an exploitable prototype without having to develop a complex web interface in HTML/CSS/JavaScript.

However, if you already have experience with other frameworks and wish to build a more advanced architecture, you are free to do so.

3. Think of your solution as a lightweight full stack system

Even if this project remains pedagogical, it is recommended to think of your prototype as a lightweight full stack architecture in which several layers cooperate:

- a user input / interface layer
- an application logic layer
- a RAG / agents / routing layer
- possibly an API layer
- a storage / cache / vector database layer

This approach will help you:

- better organize your code,
- separate responsibilities,
- simplify testing,

- and bring your project closer to a real professional architecture.

Even if you do not develop a complex front-end, you must think about how a user interacts with the system:

- how they submit a business case,
- how they visualize intermediate analyses,
- how they read the final roadmap,
- how they rerun or compare several analyses.

4. Structure your code properly

Organize your project into clear and separate modules. Avoid putting everything in a single notebook or in a single disorganized script.
A professional Data Science / GenAI project is not limited to a monolithic notebook. You must structure your solution like a mini software product, with a clear, readable, and maintainable architecture.

Separate responsibilities:
For example, you can organize your project around modules or folders such as:

- data_ingestion/ for loading documents
- chunking_embeddings/ for chunking and embeddings
- vector_store/ for creating / loading the vector database
- retrieval/ for RAG logic
- agents/ for prompts and agent logic
- routing/ for model selection
- llm_clients/ for Gemini / Llama / other calls
- cache/ for result reuse mechanisms
- app/ for the Streamlit interface or another one
- api/ for endpoints if you expose an API
- utils/ for shared functions

A notebook can be kept for:

- exploration,
- first tests,
- or experimental validation,

but the final pipeline must be executable without depending on a disorganized notebook.

This structuring:

- facilitates group work,
- improves readability,
- strengthens reproducibility,
- and prepares you for more professional development practices.

5. Version your work regularly with Git (recommended)

Versioning is an essential good practice. Using Git allows you to:

- track your progress,
- go back in case of error,
- avoid loss of work,
- work efficiently in a team,
- structure collaboration.

You must make regular, explicit, and coherent commits, reflecting the real progress of the project, for example:

- adding corpus ingestion,
- creating the retriever,
- adding the Planner agent,
- implementing routing,
- improving the generation prompt,
- integrating the interface,
- adding the cache,
- API tests, etc.

A clean, active, and structured Git repository is a strong indicator of professional maturity. If you work in pairs, it is strongly recommended to use:

- branches,
- clear commit messages,
- possibly pull requests / merges if your organization allows it.

6. Design an independent, exploitable, and decision-oriented dashboard

Your interface or dashboard must not be thought of as a simple visual bonus. It must be designed as an autonomous decision-support tool, enabling a user to understand and exploit the system.

The dashboard can be developed with:

- Streamlit (recommended),
- Dash / Plotly,
- or any other appropriate framework.

Your interface should ideally make it possible to:

- load or enter a business case,
- launch the analysis,
- display the information retrieved by the RAG,
- present the agents' results,
- visualize the final roadmap,
- possibly compare several results,
- show which model was used at each stage,
- make outputs readable for a non-technical user.

Important: The dashboard must be a central component of the solution if you choose to develop an interactive prototype. The point is not only to display text, but to propose an interface exploitable by a business profile or an evaluator.
The final user must not need to manually execute code in a notebook in order to use your system.

7. Industrialization via API: strongly recommended to go further (optional)

To bring your project closer to a real professional use case, it is recommended to industrialize your solution by exposing certain functionalities through a REST API.

The idea is not to make the whole system run only in a notebook or directly in the dashboard, but to create an intermediate service layer.

Why expose an API?
An API makes it possible to:
- separate the front-end from the AI engine,
- make the solution more modular,
- facilitate independent testing,
- prepare an architecture closer to the real world,
- simulate future integration with other applications.

The API could include at minimum:
- POST /analyze
  receives a business case and returns a structured analysis
- POST /generate-roadmap
  receives the analysis elements and returns the final roadmap
- GET /health
  allows checking that the service is working
- possibly GET /model-info or GET /pipeline-info
  to indicate the available models or the active configuration

Error handling
You must plan for:
- input validation,
- handling of missing fields,
- explicit error messages,
- coherent HTTP codes.

Minimal documentation
The API must be documented, at least via:
- a README,
- or auto-generated documentation if you use FastAPI.

8. Test your API independently from the dashboard

If you develop an API, make sure it works properly before integrating it into the interface.

Too often, projects fail at the end because:
- the API was never seriously tested,
- endpoints were never validated on their own,
- the dashboard artificially compensates for backend issues.

Before integration into the front-end, you must test your endpoints with:
- Postman,
- curl,
- a Python script,
- or any other equivalent tool.

Make sure that:
- inputs are correctly received,
- responses are correct,

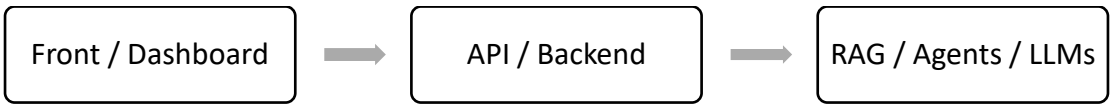
- errors are properly handled,
- outputs are stable.

The dashboard must only be a presentation layer. It must not compensate for backend weaknesses.

9. Think of your interface and your backend as two distinct layers

In a more realistic architecture, the dashboard should ideally call the API, rather than directly loading the model, the vector database, or the prompts.

This separation makes it possible to reproduce a more professional architecture:



Even if you do not implement this entire architecture completely, you must at least think about this conceptual separation.

10. Adopt a professional posture in your technical choices

This project also evaluates your ability to work like an AI engineer or a Data / GenAI consultant.

Your report, your code, and your demonstration must reflect an approach that is:

- rigorous,
- argued,
- structured,
- reproducible.

You must explain and justify your decisions, for example:

- why a given development environment was chosen,
- why Streamlit or another framework was selected,
- why an API was implemented or not,
- why a given agent architecture was adopted,
- why certain LLM calls were mutualized,
- how you reduced cost and credit consumption,
- what trade-offs you made between performance, simplicity, speed, and quality.

Your added value is not measured only by the quality of the generated text, but by your ability to produce a coherent, defensible, maintainable, and exploitable solution.

The objective is not only to generate a roadmap or obtain a convincing output.
You must demonstrate your ability to build a complete solution, including:

- a clear AI architecture,
- a functional RAG pipeline,
- specialized agents,
- multi-LLM routing,
- an optimization logic,

- a usable interface or prototype,
- and, if possible, an architecture closer to a real software system.

Industrialization through API is a very relevant extension for groups wishing to go further and highlight an approach closer to real professional practices.

Detailed Functional Requirements (FR)

FR1: Understanding the business case and the transformation purpose

The system must be able to analyze a business case written in natural language in order to extract its essential elements: context, stakes, objectives, constraints, opportunities, and transformation needs. It must also be able to identify the strategic purpose of the initiative, that is, the rationale of the transformation: solving a problem, meeting a need, seizing an opportunity, or addressing external expectations.

In particular, the system must be able to answer the question **Why transform?**, by analyzing the main transformation drivers, such as:

- customer pressure,
- competition,
- the emergence of new technologies,
- market evolution,
- performance, innovation, or agility needs.

FR2: Acquisition, preparation, and structuring of the documentary corpus

You must implement a pipeline allowing you to load, clean, structure, and exploit the documents provided in the digital transformation corpus. Depending on your technical choices, this pipeline must include:

- file loading,
- text extraction,
- chunking,
- embedding generation,
- then storage in a local vector database (for example FAISS or Chroma).

You must justify your choices of chunking, embeddings, and vector database, and explain how these choices influence retrieval quality.

Corpus preprocessing must be thought of in an optimization logic: the shared corpus must be loaded, chunked, vectorized, and indexed once, then reused. Embeddings and the vector database must not be recalculated at each execution; they must be saved and reloaded.

FR3: Implementation of the RAG pipeline

You must implement a functional Retrieval-Augmented Generation pipeline, capable of:

- searching for the most relevant passages in the corpus,
- injecting this context into prompts,
- generating responses grounded in the provided documents.

The system must demonstrate that the generated outputs genuinely rely on corpus documents and not on free, non-contextualized generation. You must therefore design a coherent retrieval mechanism and explain how it improves the reliability of responses.

The system must also limit the injected context to genuinely useful passages in order to avoid sending the entire corpus at every stage.

FR4: Explicit exploitation of digital transformation frameworks

Your system must be able to explicitly exploit the conceptual frameworks provided in the corpus, in particular:

- the **Why transform / What to transform / How to transform** framework,
- the **Digital Transformation Canvas** and its 7 action fields,
- the more recent academic framework structured around the strategic purpose, operational pillars, created value, and potential pitfalls.

The system must be able to answer **What to transform?** by relying both on:

- the IMD/Cisco framework categories,
- the 7 fields of the Digital Transformation Canvas,
- and the operational pillars **Process / People / Platform / Partners** of the academic framework.

The generated analysis must clearly show that these frameworks have been mobilized to structure reasoning and recommendations. The system must be capable of producing a multi-framework analysis articulated around:

- the **Why / What / How** logic,
- the **7 action fields**,
- the strategic purpose, operational pillars, value, and risks.

FR5: Implementation of a multi-agent system

You must develop a system composed of several specialized agents, each with a clear role in the overall reasoning. The system must include at minimum:

- a **Planner Agent** to analyze the business case, identify the stakes, structure the problem, and organize an initial Why / What / How reading;
- a **Retrieval / Framework Agent** to search for the most relevant passages, classify elements by framework, and prepare the context injected into other agents;
- a **Canvas Analysis Agent** to analyze the company according to the 7 fields of the Digital Transformation Canvas;
- a **Strategist Agent** to integrate the strategic purpose, mobilize the operational pillars, articulate the expected value and risks, and propose a strategic trajectory;
- a **Roadmap Generator Agent** to transform the analysis and the strategy into a structured roadmap.

The addition of an **Evaluator Agent** is strongly recommended. This agent must verify the quality, consistency, and completeness of the final result.

Each agent must be defined by a role, a clear instruction, identified inputs, an expected output.

FR6: Proposal of initiatives and generation of a roadmap

Based on the produced analysis, the system must generate concrete transformation initiatives. These initiatives must be explicitly linked to dimensions of the corpus and the frameworks mobilized. The system is also encouraged to identify quick wins and propose a realistic transformation scope.

The system must then produce a structured, exploitable, and contextualized digital transformation roadmap for the analyzed company.

This roadmap must include, as much as possible:

- objectives,
- initiatives,
- priorities,
- milestones,
- duration or time horizon,
- a level of effort or a budget estimate,
- monitoring or performance indicators.

The final output must not be simple free text, but a structured and readable deliverable, of the roadmap steering type.

FR7: Critical evaluation of the result

The system must integrate a final evaluation or critique stage in order to identify:

- inconsistencies,
- omissions,
- weak points,
- risks or under-treated elements.

This stage may be handled by an Evaluator Agent or by an equivalent mechanism. It must contribute to improving the robustness and credibility of the system.

FR8: Implementation of Multi-LLM and routing

You must use at least two different language models in your system, with an explicit model selection logic depending on the task. Three models are recommended if possible:

- a fast and low-cost model,
- a more performant model for complex tasks,
- a local open-source model for certain repetitive or experimental tasks.

Example of a possible organization:

- **Gemini Flash 2.5**: reformulation, synthesis, extraction, first draft;
- **Gemini Pro**: complex reasoning, strategic analysis, final generation;
- **Local Llama**: experimentation, low-criticality processing, local fallback.

The routing mechanism must be based on explicit criteria:

- task type,
- complexity,
- cost,
- latency,

- criticality of the expected output,
- or local / remote mode.

You must clearly explain in your report:
- which models are used,
- at which stages,
- and why this strategy represents a good trade-off between quality, cost, and speed.

FR9: Optimization of LLM calls and reduction of credit consumption

The expected system must not only be relevant; it must be optimized.
Your system must integrate an optimization logic in order to limit unnecessary calls to generation APIs. This includes in particular:
- saving and reusing the vector database,
- mutualizing prompt templates,
- limiting injected context,
- using lighter models for certain tasks,
- possibly a cache mechanism to reuse some intermediate results.

Agent prompts must be structured as reusable templates. Relevant intermediate outputs may be cached, for example:
- retrieval results,
- partial syntheses,
- repeated analyses,
- roadmap drafts.

A two-stage architecture is recommended:
- first draft with a more economical model,
- final refinement with a more powerful model.

All agents do not necessarily need to be called in every situation. A conditional logic is encouraged to reduce the overall cost.

You must show that you have thought about the system's efficiency, and not only its functional correctness.

FR10: Application prototype / user interface

You must develop a functional prototype making it possible to use the system without relying only on a notebook.

The interface must at minimum make it possible to:
- submit a business case or an input document,
- launch the analysis,
- visualize intermediate or final results,
- display the generated roadmap clearly.

The framework is left to your choice according to your preferences and experience. **Streamlit** is strongly recommended, because it is open source, designed for data scientists, quick to implement, and does not require advanced front-end development skills. However, you may

use another environment consistent with your project (Dash, Flask, FastAPI with a front-end interface, React, etc.) if you master it better.

FR11: API and service architecture (optional but valued)

If you choose to move toward a more industrialized logic, you may develop a REST API exposing some system functionalities.

This API may include at minimum:

- POST /analyze or POST /generate-roadmap
- GET /health
- robust error handling
- minimal documentation (README, Swagger, or FastAPI documentation)

In this case, it is recommended that the interface call the API rather than directly loading the AI pipeline, in order to reproduce a more realistic **Front / API / Model** type architecture.

Additional Optimization Recommendations for Multi-LLM, Routing, and Credit Consumption Reduction

The expected system must not only be functionally relevant; it must also be optimized, both in terms of cost, latency, and execution efficiency.

In this project, you must think not only about the quality of the generated outputs, but also about how to reduce unnecessary model calls, better distribute tasks among several LLMs, and intelligently reuse already performed processing.

1. Integration of a Multi-LLM logic

You must integrate at least two different language models into your system. The objective is to avoid using the same model for all tasks, whereas some require advanced reasoning and others can be handled by a lighter model.

Three types of models are recommended if possible:

- a fast and low-cost model, adapted to simple or intermediate tasks;
- a more performant model, adapted to complex reasoning or final generation tasks;
- a local open-source model, useful for certain repetitive, experimental, or low-criticality tasks.

Example of a possible organization

- **Gemini Flash 2.5**: reformulation, synthesis, extraction, first draft
- **Gemini Pro**: (using the activated $300) complex reasoning, strategic analysis, final generation
- **Local Llama**: experimentation, low-criticality processing, local fallback

The goal is not simply to multiply models, but to use each model where it brings the most value.

Set up Free Google Cloud Credits

Free Trial Access: https://cloud.google.com/free?hl=en

New customers get $300 in free credit to explore Google Cloud products and build a proof of concept (PoC).
You will not be charged unless you explicitly upgrade to a paid account.

Conditions

To benefit from this free trial, you must:

- Use a new Google (Gmail) account
- Never have claimed the $300 credits before
- Use the credits within a 90-day period

Gemini API (Free Tier Access)

In addition to the trial credits, Google provides access to Gemini models via a free tier API (available today (2026): Google Gemini 2.5 Flash).
This allows you to:

- Test LLM capabilities
- Build AI agents

- Integrate generative AI into your project

Attention: Free Tier Limits

The free tier includes daily usage limits (which may evolve over time), typically based on:

- Number of requests per day
- Number of tokens processed
- Rate limits (requests per minute)

These limits are sufficient for: learning, prototyping, small-scale experiments

2. Setting up a routing mechanism

The system must integrate an explicit routing logic, that is, a mechanism for selecting the most appropriate model depending on the task.

Routing must be based on clear criteria, such as:

- task type,
- complexity,
- cost,
- latency,
- criticality of the expected output.

For example:

- a simple summarization task may be assigned to a fast model,
- a strategic analysis or roadmap generation task may be assigned to a more powerful model,
- certain repetitive or comparison tasks may be assigned to a local model.

In your report, you must explain:

- which models are used, at which stages of the pipeline,
- and why this strategy constitutes a good trade-off between quality, cost, and speed.

3. Offline preprocessing of the corpus

The shared documentary corpus must be processed in an offline logic. In other words, costly preparation steps must not be repeated at each execution.

The corpus must be loaded, chunked, vectorized, indexed, once only, then reused.
This approach makes it possible to reduce execution time and avoid unnecessary repeated processing.

4. Persistent vector database

Embeddings and the vector database must not be recalculated at each system launch.
You must:

- save the vector database once it has been created,
- then reload it during subsequent executions.

This makes it possible:

- to save time,
- to reduce resource consumption,
- and to make your system closer to a real professional architecture.

5. Mutualization of prompt templates

The prompts used by agents must not be manually rebuilt each time in an ad hoc way.

It is recommended to define reusable templates for:
- the Planner Agent,
- the Analysis Agent,
- the Strategist Agent,
- the Roadmap Generator,
- the Evaluator Agent.

This mutualization makes it possible:
- to ensure better system consistency,
- to simplify maintenance,
- to reduce unnecessary prompt length,
- and to limit the number of costly calls or reformulations.

6. Reduction of injected context

The system must inject into prompts only the passages genuinely useful for the current task.

You must therefore avoid:
- sending the entire corpus,
- repeating already known content,
- or unnecessarily overloading models with too broad a context.

The objective is to optimize:
- the number of consumed tokens,
- latency,
- and response quality.

7. Caching intermediate results

Relevant intermediate outputs may be cached in order to avoid recalculating the same results several times.

This may concern, for example:
- retrieval results,
- partial syntheses,
- repeated analyses on the same case,
- roadmap drafts.

Caching is strongly recommended, especially if you use free APIs or APIs limited in credits, because it makes it possible to:
- reduce call consumption,
- accelerate execution,
- and make the system more stable.

8. Two-stage generation

A two-step generation architecture is recommended:
- first draft with a more economical model,
- final refinement with a more powerful model.

This approach makes it possible:
- to limit the intensive use of the most expensive model,
- to keep good final quality,
- and to distribute the generation load more intelligently.

9. Conditional execution of agents

All agents do not necessarily need to be called in all situations.

A conditional logic is encouraged in order to reduce the overall pipeline cost.
For example:
- some simple tasks may be limited to a few stages,
- while more complex cases will mobilize all agents.

The objective is to avoid a systematically heavy orchestration when certain stages are not actually necessary.

10. Overall expected result

You must show that you have designed a system that does not only seek to produce a good response, but also to:
- better distribute tasks among models,
- reduce unnecessary calls,
- reuse already performed processing,
- and optimize the balance between performance, cost, and speed.

Your system must be both intelligent, structured, and economical.

Evaluation Criteria

The evaluation will focus on:

- understanding of the business need,
- quality of the RAG pipeline,
- relevance of the multi-agent reasoning,
- proper exploitation of the frameworks,
- consistency of the multi-LLM routing,
- level of system optimization,
- quality of the generated roadmap,
- clarity of the code, documentation, and demonstration.

This project aims to move you from a simple call to an LLM toward the design of a complete, structured, reasoned, documented, and optimized AI system.
You must not only obtain a response, but design a system capable of searching, reasoning, structuring, arbitrating, and producing a credible strategic deliverable.

Deliverables

You must submit your deliverables on **BOOSTCAMP**. A **Git / GitHub repository is strongly recommended** for versioning, collaboration, and work traceability (but may remain optional if specified by the instructor).

Expected deliverables

- **Source code of the functional solution**
- **Project report**
- **Project presentation slides**

Source code
The code must be:

- functional,
- structured,
- commented,
- and clearly organized.

Additional technical documentation in an appendix or in a README is recommended.

Project report
The report must present:

- the context and objective,
- the overall architecture,
- the RAG pipeline,
- the agents and their roles,
- the Multi-LLM logic and routing,
- the optimization strategies,
- the obtained results,
- a critical discussion of limitations and avenues for improvement.

Presentation slides

The presentation must make it possible to clearly explain:

- the problem addressed,
- the chosen architecture,
- the methodological choices,
- the prototype demonstration,
- the added value of the solution.

Presentation and demonstration

An oral presentation with demonstration will take place in class during the last session of the module.

- All group members must participate in the presentation.
- The demonstration must show a concrete use case.
- The evaluation may include an individual part depending on each member's involvement in the defense and in technical mastery of the project.

GOOD LUCK!